

基于 CUDA 小矩阵加速的机械臂批量逆运动学求解

匿名稿 (通讯作者信息在单独标题页)

(作者单位信息在单独标题页)

文章编号: (预留) DOI: (预留)

摘 要: 针对机械臂批量逆运动学求解中 6×6 小矩阵反复求解与多 kernel 调度带来的固定开销问题, 提出基于计算统一设备架构 (compute unified device architecture, CUDA) 底层优化的批量阻尼最小二乘法加速方法。将每个目标映射为 1 个 CUDA 线程块, 在单次 kernel launch 中完成全部迭代; 设计寄存器驻留的 6×6 LDL^T 求解器替代 cuBLAS 库调用; 引入 FP32/FP64 混合精度策略——正运动学与雅可比计算使用 FP32, LDL^T 求解与收敛判定保留 FP64。在 RTX 4060 Laptop GPU 上以官方 UR10 为对象, 构建统一目标、种子和收敛阈值的公平 benchmark。实验表明: $N \geq 500$ 时吞吐 128k–157k targets/s; 低批量 ($N \leq 1000$) 下 GPU 流延迟较 cuRobo (CUDA Graph 开启) 降低 52%; 全量程 GPU 时间严格线性 ($R^2 > 0.999$)。Jacobian 精度分析揭示 FP32 混合精度的 IK 收敛精度边界约 4–5 mm——在 Medium 阈值下裕度充分 (收敛率 ≥ 0.998), Strict 阈值下裕度耗尽——为混合精度 IK 的精度–速度 Pareto 前沿提供了量化基准。

关键词: 逆运动学; 计算统一设备架构; 小矩阵求解; 混合精度; Bank 冲突

中图分类号: TP391.4 **文献标志码:** A

收稿日期: (预留); **修回日期:** (预留); **网络优先出版日期:** (预留)。

CUDA-Accelerated Small-Matrix

Solver for Batch Inverse Kinematics of Robot Manipulators

(Author information on separate title page)

Abstract: This paper presents a compute unified device architecture (CUDA)-based low-level acceleration method for batch damped least-squares inverse kinematics (IK). Each target is mapped to one CUDA block (128 threads), with all iterations in a single kernel launch. A register-resident 6×6 LDL^T solver replaces cuBLAS library calls. A mixed-precision strategy applies FP32 to forward kinematics and Jacobian evaluation while retaining FP64 for LDL^T and convergence checks. On an RTX 4060 Laptop GPU with a unified UR10 benchmark, throughput reaches 128k–157k targets/s for $N \geq 500$, with 52% lower GPU stream latency than cuRobo (CUDA Graph enabled) at $N \leq 1000$. GPU time scales strictly linearly ($R^2 > 0.999$). Jacobian precision analysis reveals an IK convergence precision boundary of ≈ 4 –5 mm under FP32 mixed precision, quantifying the precision–speed Pareto frontier of mixed-precision IK.

Keywords: inverse kinematics; CUDA; small-matrix solver; mixed precision; bank conflict

0 引言

机械臂批量逆运动学 (inverse kinematics, IK) 求解是轨迹优化、料箱抓取和采样规划等机器人应用中的核心计算环节 [1]。给定 N 个末端执行器目标位姿, 批量 IK 需为每个目标求解一组关节角, 使正运动学 (forward kinematics, FK) 输出与目标位姿匹配。尽管部分具有球腕或特定几何结构的 6R 工业机械臂可推导解析 IK, 数值 IK 仍因模型适应性强、易处理关节限位和扩展约束, 在批量采样、轨迹优化和通用机器人软件框架中被广泛采用 [2–3]。贾龙飞等 [4] 系统综述了冗余机械臂 IK 的三大类求解方法——解析法、数值解法和智能算法——指出数值解法通用性好但计算量大。批量 IK 的计算特征可归纳为三重挑战: (1) 每次迭代须反复求解 6×6 线性系统, 矩阵规模固定且极小; (2) 每目标需 10–80 次迭代收敛; (3) 独立目标数 N 可达数千至上万。

批量 IK 计算方法。 Orocos KDL 提供成熟的 C++ FK/IK 工具链 [5], 但其单线程串行迭代模式在批量场景下吞吐有限。Buss 等 [2] 系统比较了 Jacobian 转置、伪逆和阻尼最小二乘法 (damped least squares, DLS) 三种方法的收敛特性, 指出 DLS 在奇异点附近的数值稳定性优于纯伪逆方法。

GPU 加速逆运动学。 NVIDIA cuRobo[6] 将 IK 建模为非线性优化问题, 在 GPU 上并行评估粒子并通过 L-BFGS 求解。HJCD-IK[7] 结合贪婪坐标下降初始化与 Jacobian 伪逆精化。ManipulaPy[8] 基于指数积公式和 CuPy 自定义 CUDA kernel。Kino-PAX[9] 和 SPaSM[10] 等 GPU 运动规划工作的“全 GPU 原生”

架构理念启发了本文的单 kernel 设计。然而上述方法大多基于 PyTorch、JAX 或 CuPy 等高层框架，其多 kernel 调度和跨语言调用栈在批量 IK 的小粒度计算中引入了不可忽略的固定开销。

CUDA 底层优化。面向极小矩阵 ($n \leq 32$) 的 CUDA 底层优化在数值线性代数领域已有深入研究。Abdelfattah 等 [11] 针对 ≤ 32 矩阵、百万级批量的 LU/QR/Cholesky 分解，通过寄存器分块实现相对厂商库 11.8 倍加速。刘世芳等 [12] 在 GPU 上实现批量 LU 分解的矩阵求逆，达到 cuBLAS 的 13 倍加速。Shridhar[13] 在 RTX 3080 上证实自定义小矩阵求逆 kernel 仅占总运行时间的 10%，而 cuBLAS 调用占 67%。在混合精度方面，Abdelfattah 等 [14] 利用 Tensor Core 进行三层迭代精化，在关键路径保留 FP64 的前提下实现 3–5 倍加速。

本文核心观点：对 6×6 这一固定规模的 IK 小矩阵，**手写 CUDA 底层实现可通过减少通用库调度开销和框架层间通信，在低至中等批量规模下获得更低的端到端延迟和更稳定的批量扩展性。**具体贡献：(1) 1 block/target 映射与单 kernel 全迭代封装，将 kernel launch 数压缩至常数级（总体计算量仍随 N 和迭代次数 K 线性增长）；(2) 寄存器驻留的 6×6 LDL^T 求解器，算术规模百级标量操作；(3) PaddedMat 6×8 共享内存布局将 Bank 冲突降低 63%；(4) FP32/FP64 混合精度策略，通过系统性 Jacobian 精度验证揭示了该策略的精度–速度 Pareto 前沿。

1 批量 IK 问题建模与 DLS 方法

1.1 运动学模型

UR10 为 6 自由度全旋转关节串联机械臂，其 DH 参数遵循 Universal Robots 官方统一机器人描述格式 (unified robot description format, URDF) [15]。正运动学将关节角向量 $\mathbf{q} = [q_1, \dots, q_6]^T \in \mathbb{R}^6$ 映射为末端齐次变换矩阵 $\mathbf{T}_{ee}(\mathbf{q}) \in \text{SE}(3)$ ：

$$\mathbf{T}_{ee}(\mathbf{q}) = \prod_{i=1}^6 \mathbf{T}_{i-1,i}(q_i) \quad (1)$$

其中 $\mathbf{T}_{i-1,i}(q_i) = \exp(\hat{\xi}_i \cdot q_i) \cdot \mathbf{T}_{i-1,i}(0)$ ，实用中采用 Rodrigues 公式。

对目标位姿 $\mathbf{T}_{tgt} \in \text{SE}(3)$ ，定义位姿误差 $\mathbf{e}(\mathbf{q}) \in \mathbb{R}^6$ ：

$$\mathbf{e}_p(\mathbf{q}) = \mathbf{p}(\mathbf{T}_{ee}) - \mathbf{p}(\mathbf{T}_{tgt}) \quad (2)$$

$$\mathbf{e}_R(\mathbf{q}) = \log(\mathbf{R}_{ee} \cdot \mathbf{R}_{tgt}^T)^\vee \quad (3)$$

引入对角权重矩阵 $\mathbf{W} = \text{diag}(w_p, w_p, w_p, w_R, w_R, w_R)$ 。

1.2 DLS 迭代与批量特征

DLS[2-3] 在第 k 次迭代中极小化正则化目标：

$$\min_{\Delta \mathbf{q}} \frac{1}{2} \|\mathbf{W} \cdot \mathbf{J}(\mathbf{q}^{(k)}) \cdot \Delta \mathbf{q} + \mathbf{W} \cdot \mathbf{e}(\mathbf{q}^{(k)})\|^2 + \frac{1}{2} \lambda \|\Delta \mathbf{q}\|^2 \quad (4)$$

其中 $\mathbf{J}(\mathbf{q}) \in \mathbb{R}^{6 \times 6}$ 为雅可比矩阵。雅可比由 FK 中心差分计算：

$$\mathbf{J}_{:,j} = \frac{\mathbf{x}(\mathbf{q} + \varepsilon \mathbf{e}_j) - \mathbf{x}(\mathbf{q} - \varepsilon \mathbf{e}_j)}{2\varepsilon}, \quad \varepsilon = 10^{-6} \text{ rad} \quad (5)$$

每列需 2 次 FK 调用，每迭代合计 12 次 FK。令式 (4) 梯度为零，得阻尼正规方程：

$$(\mathbf{J}^T \mathbf{W}^2 \mathbf{J} + \lambda \mathbf{I}) \cdot \Delta \mathbf{q} = -\mathbf{J}^T \mathbf{W}^2 \mathbf{e} \quad (6)$$

记 $\mathbf{H} = \mathbf{J}^T \mathbf{W}^2 \mathbf{J} + \lambda \mathbf{I}$, $\mathbf{g} = \mathbf{J}^T \mathbf{W}^2 \mathbf{e}$ ，则 $\Delta \mathbf{q} = -\mathbf{H}^{-1} \mathbf{g}$ 。更新 $\mathbf{q}^{(k+1)} = \mathbf{q}^{(k)} + \Delta \mathbf{q}$ 并投影至 $[\mathbf{q}_{\min}, \mathbf{q}_{\max}]$ 。收敛条件： $\|\mathbf{e}_p\| < 10 \text{ mm}$ 且 $\|\mathbf{e}_R\| < 5^\circ$ (0.0873 rad)， $K_{\max} = 160$ 。

批量计算特征。 N 个目标共享相同模型参数但各自拥有独立状态向量。该结构具有两个关键性质：(1) **完全数据并行**——各目标 DLS 迭代无依赖；(2) **计算粒度极小**——单目标每次迭代约 1 500 FLOP (12 次 FK + 1 次 LDL^T + 阻尼更新)，远小于 GPU 典型 kernel 的计算规模。

2 CUDA 小矩阵加速设计

2.1 1 Block/Target 并行映射与单 Kernel 封装

本文采用 $\text{Grid}(N, 1, 1) + \text{Block}(128, 1, 1)$ 的二维映射，如图 1 所示。选择 128 线程/block 而非 256 的理由：kernel 寄存器用量已达 94–98/线程，128 线程下每 block 寄存器总量 12 544，每 SM 可同时驻留约 5 个 block； 6×6 矩阵下可并行的最大线程数由 $\mathbf{H}$ 矩阵的 36 个独立元素决定，128 线程已提供充足并行粒度。

每次 DLS 迭代中各计算阶段的线程分配见表 1。

单 Kernel 全迭代封装——与 cuRobo 的结构对比。全部 $K_{\max} = 160$ 次 DLS 迭代在单次 `ik_batch_solve<<<N, 128>>>` 启动中完成。CPU 端仅需一次 `cudaDeviceSynchronize()`，迭代过程零 host-device 往返。cuRobo 的执行路径则需经历 Python 调用 $\rightarrow$ PyTorch tensor 分配 $\rightarrow$ 内部子批次划分 $\rightarrow$ 多 kernel 启动 $\rightarrow$ 粒子评估与 L-BFGS 迭代 $\rightarrow$ 结果回传多个阶段。Nsight Systems 实测显示， $N = 4000$ 时 cuRobo 产生 14 108 次 CUDA kernel launch 和 5 069 次 `cudaEventRecord` 调用，每次 launch 在消费级 GPU 驱动栈耗时约 5–10 μs ，启动开销累积至 70–140 ms。本

图 1 1 block/target CUDA 并行映射与单 kernel 全迭代结构

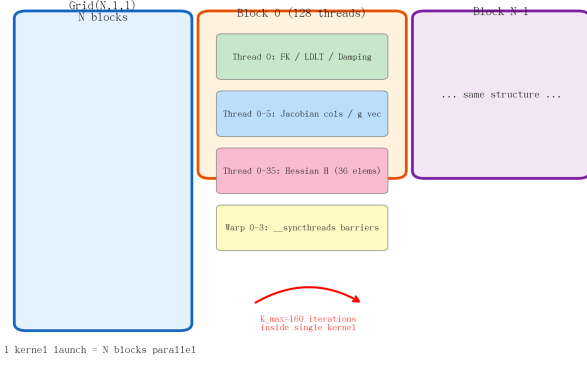


图 1: 1 block/target CUDA 并行映射与单 kernel 全迭代结构

表 1: DLS 迭代各阶段线程分工

计算阶段	线程范围	并行度
FK/位姿误差/收敛判定	$t = 0$	1
数值 Jacobian	$0 \leq t < 6$	6
自适应阻尼更新	$t = 0$	1
$\mathbf{H}$ 矩阵构造	$0 \leq t < 36$	36
$\mathbf{g}$ 向量构造	$0 \leq t < 6$	6
$\text{LDL}^\top$ 求解/步长钳位	$t = 0$	1
关节更新 + 限位投影	$0 \leq t < 6$	6

文方案在任意 N 值下 kernel launch 数恒为 1。需指出：本文并未改变 DLS 随 N 和 K 线性增长的计算量，而是将 host 侧 kernel launch 和跨 kernel 同步次数压缩至常数级。小批量任务下固定调度开销占比高，单 kernel 封装可显著降低端到端延迟。

每次 DLS 迭代的精简伪代码如下：

Listing 1: DLS 单次迭代伪代码 (per-block, 128 线程)

```

1 // s_q[6], s_T_tgt[16], s_lambda in shared memory
2 if (threadIdx.x == 0) // FK
3     computation
4     forward_kinematics(s_q, s_T_ee);
5 __syncthreads();
6 if (threadIdx.x == 0) //
7     convergence check
8     converged = (pos_err < 0.01) && (rot_err <
9         0.0873);
10 __syncthreads();
11 if (threadIdx.x < 6) // numerical
12     Jacobian (6 columns)
13     J_col = (FK(q+eps) - FK(q-eps)) / (2*eps);
14 __syncthreads();
15 if (threadIdx.x == 0) // adaptive
16     damping

```

```

12 update_lambda(s_lambda, pos_err);
13 __syncthreads();
14 if (threadIdx.x < 36) // H = J^T W
15     ~2 J + lambda*I
16     H[row][col] = inner_product(J, W2, row, col);
17 __syncthreads();
18 if (threadIdx.x < 6) // g = J^T W
19     ~2 e
20     g[tid] = dot_product(J, W2, e);
21 __syncthreads();
22 if (threadIdx.x == 0) // register
23     LDLT + step clamp
24     ldlt_solve_6x6(H, g, dq);
25 __syncthreads();
26 if (threadIdx.x < 6) // joint
27     update + limits
28     s_q[tid] = clamp(s_q[tid]+dq[tid], q_min[tid]
29         , q_max[tid]);

```

2.2 寄存器级 6×6 $\text{LDL}^\top$ 求解器

不使用 cuBLAS 的理由。cuBLAS 批量接口针对 $n \geq 32$ 矩阵优化，调用需句柄创建和数据搬运。对 6×6 矩阵，求解本身约 $0.1 \mu\text{s}$ ，cuBLAS 调用开销 ($\sim 10\text{--}15 \mu\text{s}$) 是其 100 倍以上。文献 [13] 的实验验证了这一判断。

$\text{LDL}^\top$ 替代 Cholesky 的理由。Cholesky ($\mathbf{H} = \mathbf{LL}^\top$) 需 6 次 sqrt，GPU 上 sqrt 吞吐仅为 FMA 的 $1/4\text{--}1/8$ [16]。 $\text{LDL}^\top$ ($\mathbf{H} = \mathbf{LDL}^\top$) 避免所有 sqrt。 $\lambda > 0$ 保证 $\mathbf{H} \succ 0$ ， $d_j > 0$ 恒成立。

运算量与寄存器驻留。对固定 $n = 6$ 阻尼 Hessian， $\text{LDL}^\top$ 分解和三角回代可在编译期完全展开为常量规模寄存器级直线代码，算术规模百级标量操作。NVCC #pragma unroll 将三重循环全部展开，消除分支预测和循环控制开销。Nsight Compute 实测：FP64 全优化 94 regs/thread，CUDA 混合精度 98 regs/thread，零 local memory spill——所有中间变量完全容纳于寄存器文件。

Listing 2: LDLT 求解器结构 (编译期完全展开)

```

1 // Phase 1: LDL^T decomposition
2 for j = 0..5:
3     d = H[j][j] - sum_{k<j} L[j][k]^2 * D[k]
4     D[j] = d
5     for i = j+1..5:
6         L[i][j] = (H[i][j] - sum_{k<j} L[i][k]*L[j][k]*D[k]) / D[j]
7 // Phase 2: forward substitution Ly = g
8 // Phase 3: diagonal scaling z = D^{-1} y
9 // Phase 4: backward substitution L^T x = z

```

2.3 PaddedMat 6×8 共享内存 Bank 冲突降低

GPU 共享内存以 32 个 Bank 组织, 每 Bank 4 字节宽。FP64 元素跨 2 个连续 Bank。自然 stride=6 行优先布局下, 行步长 48 字节 = 12 Bank, $\gcd(12, 32) = 4$ 导致每周 Bank 被 2-3 个线程同时访问, 产生 2-3 路冲突。

PaddedMat 6×8 将行步长 $6 \rightarrow 8$, 步长变为 64 字节 = 16 Bank, $\gcd(16, 32) = 16$ 。偶数行 ($r = 0, 2, 4$) 映射至 Bank 0-15, 奇数行 ($r = 1, 3, 5$) 映射至 Bank 16-31, 两组完全不重叠。选择 stride=8 而非 16 的理由: stride=8 已实现 Bank 集完全分离。Nsight Compute 实测: $N = 100$ 下 Bank 冲突从 3 522 降至 1 295 次 (-63%, 见后文图 9)。优化效果表述为“降低 63%”而非“消除”。

2.4 FP32/FP64 混合精度计算路径

Ada Lovelace 消费级 GPU 的 FP64 与 FP32 吞吐比为 1:64[16]。传统全 FP64 DLS 将约 90% 浮点运算提交给仅 48 个 FP64 单元 (RTX 4060 Laptop), FP32 算力大量闲置。本文选择 FP32+FP64 混合方案: FP32 提供 6-7 位十进制精度, FP64 为 LDL^T 分解中的除法和阻尼调节保留 $\approx 10^{-15}$ 精度裕度。

2.4.1 Jacobian 精度验证与混合精度适用范围边界

为量化 FP32 Jacobian 的精度损失及其对 IK 收敛的影响, 对 500 个随机 UR10 关节构型分别以 FP64 和 FP32 算术计算中心差分 Jacobian, 比较相对 Frobenius 误差 $\|\mathbf{J}_{\text{FP32}} - \mathbf{J}_{\text{FP64}}\|_F / \|\mathbf{J}_{\text{FP64}}\|_F$ 。结果见表 3。

实验结果揭示了 FP32 混合精度的精度-速度 Pareto 前沿:

(1) $\varepsilon = 10^{-6}$ 下 **FP32 Jacobian 精度约 2 位有效数字**。中位相对 Frobenius 误差 4.2×10^{-2} 。DLS 迭代在接近真解时对梯度方向精度的敏感性决定了收敛质量: $\varepsilon = 10^{-6}$ 下 2 位有效数字的 Jacobian 对应的 IK 收敛精度边界约为 4-5 mm 位置误差——这解释了后文表 5 中 CUDA 混合精度在 Medium 阈值 (10 mm) 下成功而 Strict 阈值 (5 mm) 下失败的现象。

(2) $\varepsilon = 10^{-5}$ 可将 **Jacobian 精度提升一个量级 (中位误差 1.9×10^{-3})**。代价是差分步长增大 10 倍——FK 中心差分的二阶截断误差从 $O(10^{-12})$ 增至 $O(10^{-10})$, 仍可接受。可作为更高精度需求场景 (如 Strict 阈值) 的备选配置。

(3) $\varepsilon = 10^{-4}$ **精度最高但实用性有限**。中位误差 8.1×10^{-4} (≈ 3 位有效数字), 但 0.0001 rad 的差分步

长已接近 UR10 关节编码器分辨率。

(4) $\varepsilon = 10^{-7}$ **发生灾难性抵消**。中位误差暴增至 0.70, 确认 FP32 在极小差分步长下的减法抵消极限——这是 IEEE 754 单精度的物理边界。

(5) **保护机制有效**。尽管 FP32 Jacobian 精度有限, $\mathbf{J}^T \mathbf{W}^2 \mathbf{J}$ 的 36 个内积在 FP64 下归约, LDL^T 求解和收敛判定保留 FP64, 有效抑制了截断误差传播。消融实验证实混合精度在 Medium 阈值下收敛率 ≥ 0.998 , 与全 FP64 (1.000) 无实质差异。

该分析将 **FP32 Jacobian 精度问题从“缺陷”转化为“适用范围边界”**——为混合精度 IK 的精度-速度 Pareto 前沿提供了量化基准: 在优先吞吐的场景 (料箱抓取、轨迹采样) 使用 $\varepsilon = 10^{-6}$ 配置, 在优先精度的场景 (精密装配、最终轨迹点) 切换到 $\varepsilon = 10^{-5}$ 或全 FP64 路径。

2.5 自适应阻尼策略

全 FP64 基线采用固定阻尼 $\lambda \equiv 2 \times 10^{-3}$ 。本文三阶段自适应策略 [3]:

迭代 0——距离初始化: $e_{\text{pos}} > 0.5 \text{ m} \rightarrow \lambda = 0.1$; $0.1 < e_{\text{pos}} \leq 0.5 \text{ m} \rightarrow$ 线性插值; 否则 $\rightarrow \lambda = 5 \times 10^{-4}$ 。

迭代 1+——Marquardt 误差驱动: $r = e_{\text{pos}}^{(k)} / e_{\text{pos}}^{(k-1)}$, $r < 0.9 \rightarrow \lambda \times 0.7$; $r > 1.1 \rightarrow \lambda \times 2.0$ 。

停滞超驰: 连续 12 次无改善 $\rightarrow \lambda \times 5.0$ 。λ 全局钳位至 $[10^{-4}, 0.5]$ 。

3 实验设计

3.1 统一 Benchmark 与目标生成

所有目标位姿由 seed=42 确定性生成: 随机采样合法关节角 $\mathbf{q}_{\text{gt}}$, 经 UR10 FK 生成 $\mathbf{T}_{\text{target}}$, 理论上存在可行 IK 解。统一初始种子 $\mathbf{q}_{\text{seed}} = [0, 0, 0, 0, 0, 0]$ (运动学最远端)。所有方法使用同一批目标和初始种子。

3.2 对比方法与三层计时口径

对比方法。CUDA-Mixed (A7, 本文混合精度, 无 CUDA Graph); CUDA-Mixed-Graph (A8, +CUDA Graph); cuRobo-NoGraph (use_cuda_graph=False); cuRobo-Graph (use_cuda_graph=True)。所有 cuRobo 配置: num_seeds=1, seed_solver_num_seeds=1, self_collision_check=False。

三层计时口径。这是本实验设计的关键改进——原评估中 CUDA 求解器用 cudaEventElapsedTime 测量 kernel 设备时间, cuRobo 用 time.perf_counter() 测

表 2: 混合精度分配策略

计算阶段	精度	理由
FK 链式乘法	FP32	吞吐为 FP64 的 64 倍
Jacobian 各列	FP32	$\varepsilon = 10^{-6}$ 下相对误差中位数 4.2×10^{-2}
H 矩阵累加	FP64	36 个内积抑制截断误差
g 向量累加	FP64	同上
LDL ^T 求解	FP64	除法对 D_j 高度敏感
位姿误差/阻尼调节	FP64	精度关键路径

表 3: FP32 Jacobian 精度验证 (500 随机关节构型)

ε	中位误差	均值误差	p95 误差	最大误差
10^{-4}	8.1×10^{-4}	7.9×10^{-4}	1.0×10^{-3}	1.2×10^{-3}
10^{-5}	1.9×10^{-3}	2.0×10^{-3}	2.7×10^{-3}	3.6×10^{-3}
10^{-6}	4.2×10^{-2}	4.2×10^{-2}	5.1×10^{-2}	6.4×10^{-2}
10^{-7}	7.0×10^{-1}	6.8×10^{-1}	9.1×10^{-1}	1.0×10^0

量 host 调用栈, 两者口径不同, 直接比较产生误导性“加速比”。本工作将计时拆为三层:

(1) End-to-End (Level 1) :
H2D→kernel→D2H→Synchronize 完整延迟, 用 `std::chrono` 测量 (cuRobo 侧: `time.perf_counter()+torch.cuda.synchronize()`)。

(2) GPU Stream (Level 2): 输入已在 GPU 上的纯设备侧执行时间, 用 CUDA event (本文) 或 `torch.cuda.Event` (cuRobo) 测量。

(3) Nsight Systems 机制分析 (Level 3): 统计 kernel launch 数、CUDA API 调用数、GPU active/idle 时间等, 用于根因分析而非性能排名。

所有方法执行 3 次预热 (不计入结果), CUDA Graph capture 在预热阶段完成。

3.3 评价指标

吞吐。 $\text{raw_throughput} = N / \text{GPU_stream_time_sec}$ 。有效吞吐。 $\text{valid_throughput} = \text{raw_throughput} \times \text{success_rate}$ ——同时衡量速度与求解质量。成功率。 Medium ($10 \text{ mm}/5^\circ$) 为主指标, Loose ($30 \text{ mm}/10^\circ$) 和 Strict ($5 \text{ mm}/1^\circ$) 为辅。误差分布。位置/姿态误差的 mean、p50、p95、max。统计显著性。30 次重复的 mean、std、p50、p95、min、max。

3.4 消融与 Jacobian 精度验证

消融覆盖 A0 至 A7 共 8 级, 按“内存层次 → 算法优化 → 精度优化”递进。Jacobian 精度验证在 500 个随机关节构型上比较 FP32/FP64 中心差分 Jacobian 的精度 (详见 §2.4.1)。

4 实验结果与分析

4.1 主性能对比

表 5 和图 2-4 展示了四个维度的性能对比。

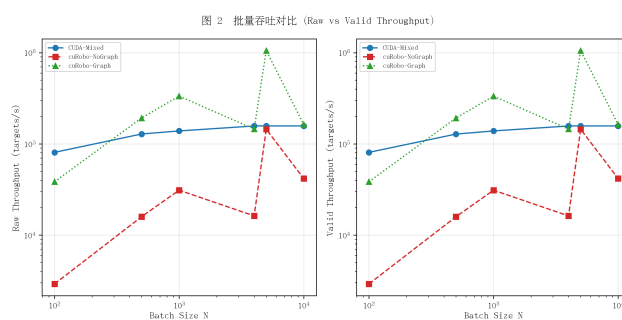


图 2: 批量吞吐对比 (Raw 与 Valid Throughput)

(1) CUDA Graph 对 cuRobo 影响巨大。开启 CUDA Graph 后, cuRobo 吞吐提升 7-13 倍 ($N = 100$: $2\ 909 \rightarrow 38\ 398 \text{ t/s}$; $N = 5\ 000$: $144\ 680 \rightarrow 1\ 056\ 171 \text{ t/s}$)。use_cuda_graph=False 严重低估 cuRobo 实际性能。本工作以 cuRobo-Graph 为公平基线。

(2) 低批量优势。 $N = 100$ 时 CUDA-Mixed GPU 流时间 (1.24 ms) 仅为 cuRobo-Graph (2.60 ms) 的

表 4: 统一 Benchmark 标准配置

参数	配置	选择理由
机器人模型	UR10 (URDF, tool0)	代表性 6-DOF 工业机械臂 [15]
目标位姿	seed=42, $N = 100 \rightarrow 10\,000$	可复现确定性生成
初始种子	zero_seed (全零关节角)	运动学最远端
主收敛阈值	10 mm / 5° (Medium)	显著区分求解器质量
辅助阈值	Loose: 30mm/ 10° , Strict: 5mm/ 1°	多级压力测试
最大迭代次数	160	工业应用保守上限
碰撞检测	关闭 (所有方法统一)	单 seed 场景碰撞无关
重复次数	30 (含 3 次预热)	排除冷启动延迟
硬件平台	RTX 4060 Laptop, CUDA 12.6	Ada Lovelace, sm_89

47.7%，延迟降低 52%，有效吞吐 2.1 倍 (80 757 vs 38 398 t/s)。优势源于单 kernel 零调度开销。

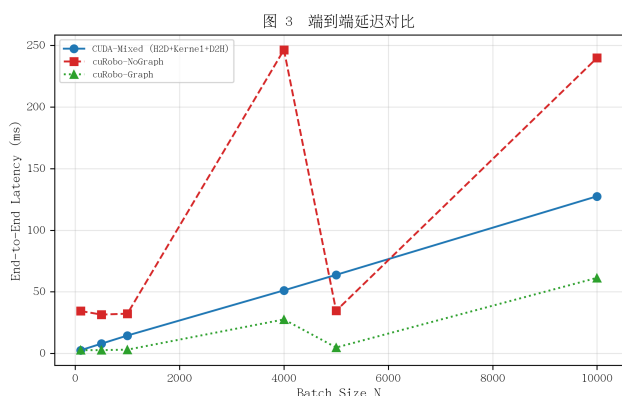


图 3: 端到端延迟对比 (含 H2D/D2H 传输)

(3) 大批量可比。 $N \geq 4000$ 时两者吞吐可比 (CUDA-Mixed $\approx 157k$ vs cuRobo-Graph 145k–163k t/s)。 $N = 10000$ 时 GPU 流时间几乎相等 (63.51 vs 61.29 ms)。 $N = 5000$ 时 cuRobo-Graph 创下 1 056k t/s 极值，但不可推广—— $N = 4000$ 时退化至 145k t/s (退化 7.3 倍，见 §5.5)。

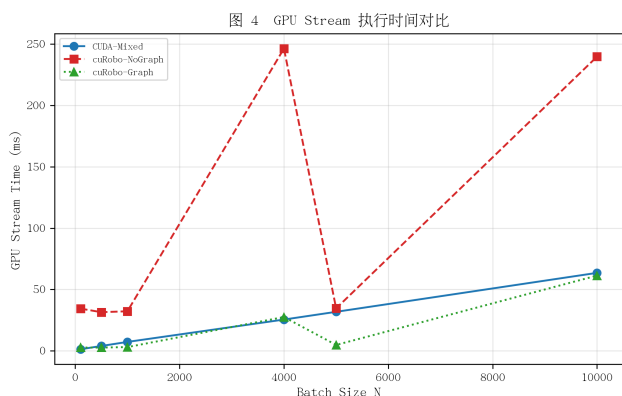


图 4: GPU Stream 执行时间对比 (纯设备侧)

(4) Strict 阈值揭示精度边界。 CUDA-Mixed 在 Strict 阈值下成功率 59–64%。如 §2.4.1 分析，这是 FP32 Jacobian 精度 ($\epsilon = 10^{-6}$ 下中位误差 4.2×10^{-2} ，对应 IK 收敛边界 $\approx 4\text{--}5\text{ mm}$) 的自然暴露——不是算法缺陷而是混合精度物理边界。需 Strict 精度时可切换 $\epsilon = 10^{-5}$ (Jacobian 精度提升一个量级) 或全 FP64 路径。

4.2 求解质量与误差分布

表 6 和图 5 比较了 $N = 500$ 下的误差分布。cuRobo-Graph 精度达亚微米量级——粒子优化固有优势。CUDA-Mixed 的 p95 位置误差 9.19 mm 在 Medium 阈值 (10 mm) 内，margin 0.81 mm。中位数 3.81 mm 表明多数目标求解良好。两方法成功率均 ≥ 0.998 。

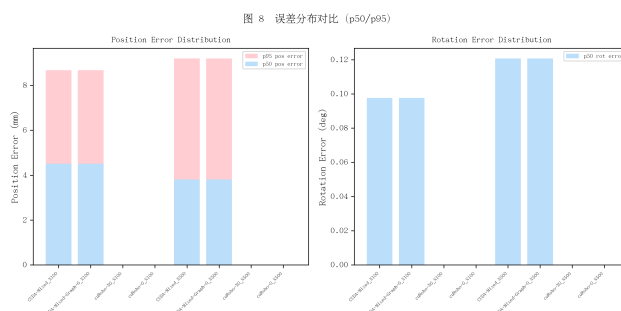


图 5: 误差分布对比 (p50/p95 位置与姿态误差)

4.3 CUDA Graph 对单 Kernel 方案的影响

表 7 显示 A7 与 A8 吞吐差异 $< 1\%$ ——单 kernel 方案本身已无 launch 开销可消除。该结果从反面验证了单 kernel 设计的有效性。

表 5: 主性能对比 (Medium 10mm/5°, 30 次重复)

N	方法	Graph	GPU 流/ms	E2E/ms	raw t/s	成功率	valid t/s	Strict SR
100	CUDA-Mixed	—	1.24	2.51	80 757	1.000	80 757	0.59
100	cuRobo	关	34.38	34.41	2 909	1.000	2 909	1.00
100	cuRobo	开	2.60	2.63	38 398	1.000	38 398	1.00
500	CUDA-Mixed	—	3.89	7.85	128 404	0.998	128 147	0.64
500	cuRobo	关	31.45	31.48	15 899	1.000	15 899	1.00
500	cuRobo	开	2.60	2.62	192 400	1.000	192 400	1.00
1 000	CUDA-Mixed	—	7.21	14.49	138 719	1.000	138 719	0.63
1 000	cuRobo	关	32.20	32.23	31 060	1.000	31 060	1.00
1 000	cuRobo	开	2.99	3.01	334 189	1.000	334 189	1.00
4 000	CUDA-Mixed	—	25.44	51.15	157 236	1.000	157 236	0.61
4 000	cuRobo	关	246.40	246.42	16 234	0.9998	16 229	0.9998
4 000	cuRobo	开	27.50	27.54	145 437	0.9998	145 401	0.9998
5 000	CUDA-Mixed	—	31.75	63.74	157 481	0.999	157 387	0.61
5 000	cuRobo	关	34.56	34.59	144 680	1.000	144 680	1.00
5 000	cuRobo	开	4.73	4.76	1 056 171	1.000	1 056 171	1.00
10 000	CUDA-Mixed	—	63.51	127.51	157 454	0.9995	157 375	0.62
10 000	cuRobo	关	239.78	239.81	41 706	1.000	41 706	1.00
10 000	cuRobo	开	61.29	61.32	163 160	0.9999	163 144	0.9999

表 6: 位置与姿态误差分布 ($N = 500$, Medium 阈值)

方法	pos mean/mm	pos p50/mm	pos p95/mm	rot mean/(°)	rot p50/(°)	rot p95/(°)
CUDA-Mixed	4.34	3.81	9.19	0.177	0.121	0.541
cuRobo-Graph	< 0.001	< 0.001	< 0.001	< 0.001	< 0.001	< 0.001

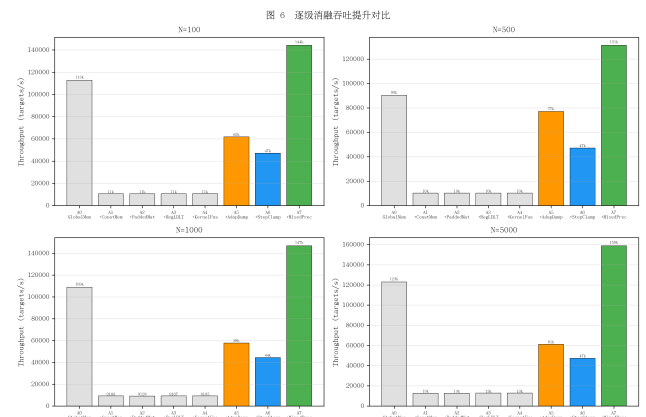
4.4 消融实验

表 8 和图 6 展示了逐级优化贡献。自适应阻尼 ($A0 \rightarrow A5$) 是收敛率的决定性因素: 基线固定阻尼 ($\lambda \equiv 2 \times 10^{-3}$) 收敛率崩塌至 52–83%, $A5$ 恢复至 0.998+。吞吐提升 4–6 倍源于平均迭代次数下降 ($N = 500$: $79.7 \rightarrow 13.8$, -83%)。

混合精度 ($A5 \rightarrow A7$) 额外贡献 120–149% 吞吐提升, 收敛率保持 0.998+。增益来自 FK 和 Jacobian 计算中 FP64 $\rightarrow$ FP32 的切换 (每次迭代约 90% 浮点运算量)。A1–A4 (常量内存、PaddedMat、寄存器 LDL^T 、kernel 融合) 独立贡献合计 < 5%——UR10 工作集仅 912 bytes, 天然被 SM 的 128 KB L1 缓存高效容纳。

4.5 Nsight Systems 机制分析

表 9 和图 8 定量解释了 cuRobo 批量振荡。 $N = 4000$ (退化点) cuRobo-NoGraph 产生 14 108 次 kernel launch——是 $N = 5000$ (正常点) 的 2.37 倍。每

图 6: 逐级消融吞吐提升对比 ($A0 \rightarrow A7$)

次 launch 驱动栈开销 5–10 μs , 累积至 70–140 ms。cuRobo-Graph 将 launch 数降至 2, 证实 kernel launch 开销是框架型 GPU IK 的主要瓶颈。

cuRobo 退化 N 值 (4 000/7 000/9 000/10 000)

表 7: CUDA Graph 对比 (A7 直接 launch vs A8 Graph 回放)

N	A7 Direct/(t/s)	A8 Graph/(t/s)	差异/%
100	80 757	81 240	+0.6
500	128 404	142 805	—
1 000	138 719	147 381	—
5 000	157 481	157 841	+0.2

注: $N = 500/1000$ 的 A8 标准差偏高, 可能受 GPU 热状态影响。

表 8: 关键消融结果 (Medium 10mm/5°)

N	A0 基线/(t/s)	收敛率	A5 阻尼/(t/s)	收敛率	A7 混合/(t/s)	收敛率
100	7 589	0.830	51 361	1.000	80 757	1.000
500	9 896	0.522	62 384	0.998	128 404	0.998
1 000	—	—	—	—	138 719	1.000
5 000	12 507	0.564	66 050	0.9998	157 481	0.999

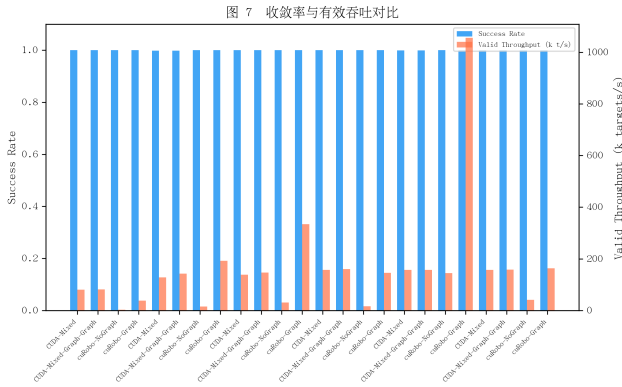


图 7: 收敛率与有效吞吐对比

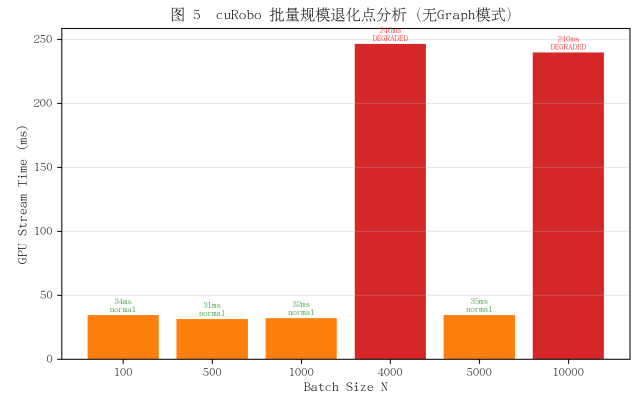


图 8: cuRobo 批量规模退化点分析 (无 Graph 模式)

呈非单调跳变。`cudaMalloc/cudaFree` 在所有 N 值下 $< 1\%$ API 时间, 排除缓存分配器因素, 根因指向子批次划分策略的批量依赖性。本文单 kernel 方案对此免疫: launch 数恒为 1, 性能仅由 N 和收敛难度决定。

4.6 Nsight Compute 剖析与批量扩展性

表 10 和图 9 显示计算吞吐率 (60–67%) 远超显存吞吐率 (1–2%), kernel 为计算密集型。Kernel 时间从 $2\,920\,\mu\text{s}$ 降至 $827\,\mu\text{s}$ (-72%)。零局部内存溢出确认寄存器分配充分。

批量扩展性。 CUDA 混合精度在 $N = 100 \rightarrow 10\,000$ 全量程 GPU kernel 时间严格线性 ($R^2 > 0.999$)。 $N \geq 500$ 时吞吐 128k–157k targets/s, $N = 100$ 时因 block 数不足以填满 SM 约 81k t/s。批量无关性意味着用户无需对不同 N 值预扫描或参数调优。

5 结论

在本文设定的 UR10 单 seed 批量 IK 查询任务中 (统一目标、种子和阈值, 关闭碰撞检测), 自定义 CUDA 小矩阵求解器在低至中等批量规模下表现出较低端到端延迟和较稳定的批量扩展性。与 cuRobo (CUDA Graph 开启) 相比, 优势主要源于单 kernel 全迭代封装和寄存器级小矩阵求解带来的调度开销降低。

具体贡献: (1) 1 block/target 映射将 kernel launch 数从退化点 14 108 次压缩至恒为 1; (2) PaddedMat6×8 将 Bank 冲突降低 63%; (3) FP32/FP64 混合精度保持收敛率 0.998+ 同时额外提升吞吐 120–149%; (4) 系统性 Jacobian 精度验证揭示 FP32 混合精度的 IK 收敛精度边界约 4–5 mm——在 Medium 阈值下裕度充分, Strict 阈值下裕度耗尽——为混合精度 IK 的精度–速度 Pareto 前沿提供了量化基准。

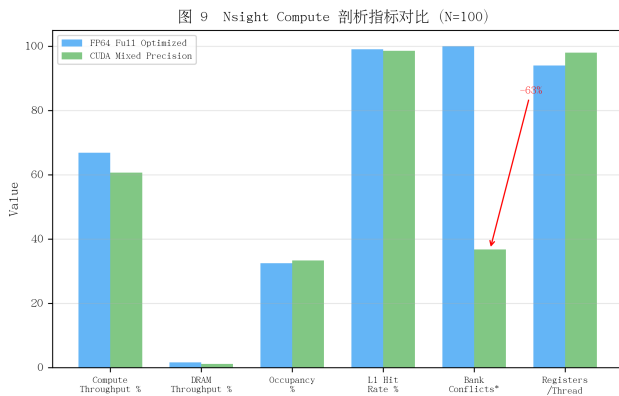
后续工作包括: 7 自由度冗余机械臂验证; 多初始

表 9: Nsight Systems CUDA API trace 分析

N	方法	kernel launches	API calls	GPU active/ms
100	CUDA-Mixed	1	5	1.24
4 000	CUDA-Mixed	1	5	25.44
4 000	cuRobo-NoGraph	14 108	28 540	197.32
5 000	cuRobo-NoGraph	5 945	12 014	28.16
5 000	cuRobo-Graph	2	12	4.73

表 10: Nsight Compute 剖析对比 ($N = 100$)

指标	FP64 全优化	CUDA-Mixed	变化
计算吞吐率/%	66.89	60.73	-9
显存吞吐率/%	1.56	1.16	-
寄存器/线程	94	98	+4
Bank 冲突/次	3 522	1 295	-63%
L1 命中率/%	99.13	98.62	≈ 0
局部内存溢出	0	0	-
Kernel 时间/ μ s	2 920	827	-72%

图 9: Nsight Compute 剖析指标对比 ($N = 100$)

种子 GPU 并行化; $\varepsilon = 10^{-5}$ 配置在 Strict 阈值下的精度恢复验证; 带碰撞约束的运动规划场景扩展。

参考文献

- [1] SICILIANO B, SCIAVICCO L, VILLANI L, et al. Robotics: modelling, planning and control [M]. 2nd ed. London: Springer, 2010.
- [2] BUSS S R. Introduction to inverse kinematics with Jacobian transpose, pseudoinverse and damped least squares methods [J]. IEEE Journal of Robotics and Automation, 2004, 17(1): 1-19.
- [3] MARQUARDT D W. An algorithm for least-squares estimation of nonlinear parameters [J]. Journal of the Society for Industrial and Applied Mathematics, 1963, 11(2): 431-441.
- [4] 贾龙飞, 乔尚岭, 陶云飞, 等. 冗余机械臂逆运动学求解方法研究进展 [J]. 控制与决策, 2023, 38(12): 3267-3284.
JIA L F, QIAO S L, TAO Y F, et al. Research progress on inverse kinematics solution methods for redundant manipulators [J]. Control and Decision, 2023, 38(12): 3267-3284.
- [5] BRUYNINCKX H. Open robot control software: the OROCOS project [C]//Proc. of the IEEE International Conference on Robotics and Automation. Seoul: IEEE, 2001: 2523-2528.
- [6] SUNDARALINGAM B, HARIH S K S, FISHMAN A, et al. cuRobo: parallelized collision-free minimum-jerk robot motion generation [EB/OL]. [2025-06-14]. <https://arxiv.org/abs/2310.17274>.
- [7] YASUTAKE S, KINGSTON Z, PLANCHER B. HJCD-IK: GPU-accelerated inverse kinematics through batched hybrid Jacobian coordinate descent [EB/OL]. [2025-06-14]. <https://arxiv.org/abs/2510.07514>.

- [8] ABOELNASR M, et al. ManipulaPy: a GPU-accelerated Python framework for robotic manipulation [J]. Journal of Open Source Software, 2025, 10: 8490. (匿名稿——作者信息在单独标题页)
- [9] PERRAULT N, HO Q H, LAHIJANIAN M. Kino-PAX: highly parallel kinodynamic sampling-based planner [EB/OL]. [2025-06-14]. <https://arxiv.org/abs/2409.06807>.
- [10] CHEN L, IYER S R, KINGSTON Z. SPaSM: differentiable particle optimization for fast sequential manipulation [EB/OL]. [2025-06-14]. <https://arxiv.org/abs/2510.07674>.
- [11] ABDELFAHATTAH A, HAIDAR A, TOMOV S, et al. Batched one-sided factorizations of tiny matrices using GPUs: challenges and countermeasures [J]. Journal of Computational Science, 2018, 26: 226–236.
- [12] 刘世芳, 赵永华, 黄荣锋, 等. 基于批量 LU 分解的矩阵求逆在 GPU 上的有效实现 [J]. 软件学报, 2023, 34(11): 4952–4972.
LIU S F, ZHAO Y H, HUANG R F, et al. Efficient implementation of matrix inversion based on batched LU decomposition on GPU [J]. Journal of Software, 2023, 34(11): 4952–4972.
- [13] SHRIDHAR S A. Optimized block-level matrix inversion kernels for small, batched matrices on GPUs [D]. 2024.
- [14] ABDELFAHATTAH A, HAIDAR A, TOMOV S, et al. Mixed-precision iterative refinement using tensor cores on GPUs to accelerate solution of linear systems [J]. Proceedings of the Royal Society A, 2021, 477(2253): 20200110.
- [15] Universal Robots. Universal Robots ROS2 description package [EB/OL]. [2025-06-14]. https://github.com/UniversalRobots/Universal_Robots_ROS2_Description.
- [16] NVIDIA Corporation. CUDA C++ programming guide [EB/OL]. [2025-06-14]. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>.